

2 marks

a) Define the following terms:

i) Database

ii) DBMS

Ans. i. Database is a collection of information or facts that is organised so that it can be easily processed, managed, controlled, accessed or updated.

ii. A Database Management System (DBMS) is software that allows users to store, organize, manage, and retrieve data efficiently from a database.

Example : Oracle, MySQL, SQL Server.

b) List any four codd's rules.

Ans. Rule 0 : Foundation rule – A system must qualify as a relational database management system (RDBMS).

Rule 1 : Information rule – All information in the database must be represented in tables.

Rule 2 : Guaranteed Access rule – Each piece of data must be accessible with a combination of table name, primary key and column name.

Rule 3 : Systematic Treatment of NULL values – Null values must be supported and treated systematically.

c) Explain subquery concept with example.

Ans. A **subquery** (or inner query) is a query that is nested inside another SQL query (the outer query).

The inner query executes first, and its result is then passed to the outer query, which uses that result to complete its own operation.

Example :

SELECT Name, Salary FROM Employees WHERE Salary > (SELECT AVG(Salary) FROM Employees);

d) Define

i) Primary Key

ii) Candidate key

Ans. i. A Primary Key is a specific field (or combination of fields) in a relational database table that uniquely identifies each record (row) in that table.

ii. A Candidate Key is a column, or a set of columns, in a table that can uniquely identify any row (tuple) without extraneous data.

In simple terms, it is a potential Primary Key.

e) List any two DDL commands with syntax.

Ans. i. Create : CREATE TABLE table_name(column1 datatype, column2 datatype);

ii. Drop : DROP TABLE table_name;

f) Draw the block structure of PL-SQL.

Ans. DECLARE

--Declaration section (optional)

BEGIN

--Execution section (mandatory)

EXCEPTION

--Exception handling (optional)

g) State syntax of while loop command.

Ans. WHILE condition LOOP

EXIT WHEN exit_condition;

END LOOP;

4 marks

a) Explain not null and check constraint with example.

Ans. 1. NOT NULL Constraint

The NOT NULL constraint is a simple yet powerful rule that ensures a column must contain a value and cannot be empty (i.e., it cannot contain NULL).

2. CHECK Constraint

The CHECK constraint is used to define a condition that every value entered into a column must satisfy.

Example :

```
CREATE TABLE Student(roll_no INT, name VARCHAR(40) NOT NULL, age INT, check(age > 18));
```

b) Explain any four string functions with example.

Ans. i. UPPER(string) : Returns all the characters in the string to uppercase.

e.g. SELECT UPPER('hello');

ii. LOWER(string) : Returns all the characters in the string to lowercase.

e.g. SELECT LOWER('HELLO');

iii. LENGTH(string) : Returns the number of characters in the string.

e.g. SELECT LENGTH('Hello');

iv. CONCAT(string1, string2) : Joins two strings.

e.g. SELECT CONCAT('SQL' , 'Functions');

c) Describe Grant and Revoke commands with example.

Ans. i. GRANT command is used to give privileges to a user on specific objects.

Syntax :

```
GRANT (INSERT/UPDATE/DELETE) ON object_name TO user_name;
```

Example :

```
GRANT INSERT, UPDATE ON employee TO Jay;
```

ii. REVOKE command is used to withdraw the privileges from a user on specific objects.

Syntax :

```
REVOKE (INSERT/UPDATE/DELETE) ON object_name FROM user_name;
```

Example :

```
REVOKE INSERT ON employee FROM Jay;
```

d) Explain trigger with suitable example.

Ans. A trigger is a stored PL/SQL block that automatically executes (fires) when a specific event occurs on a table or view such as INSERT, UPDATE or DELETE. They are often used to enforce business rules, maintain audit trails or automatically modify tables.

Example :

```
CREATE OR REPLACE TRIGGER sal_update_trg
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
INSERT INTO emp_audit(emp_id, old_salary, new_salary, change_date) VALUES (:OLD.emp_id, :OLD.salary,
:NEW.salary, SYSDATE);
END;
/
```

a) Explain views with example.

Ans. A View is a virtual table based on the result-set of a SQL query.

It does not store data itself; instead, it is stored in the database as a predefined query. When a user queries a View, the underlying query is executed, and the resulting data is presented as if it were a physical table.

Syntax :

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name;
```

Example :

```
CREATE VIEW stud AS
SELECT name, marks
FROM Student;
```

b) Explain properties of transaction.

Ans. ACID properties are a set of key principles that ensure reliability and consistency of database transactions.

1. Atomicity : A transaction is all or nothing. Either all operations in a transaction are executed or none are.
e.g. Transferring money between accounts either both debit and credit happens or neither.

2. Consistency : A transaction takes the database from one valid state to another, maintaining all integrity constraints.

e.g. After a transaction, total account balances remain correct.

3. Isolation : Transactions are executed independently of others. Intermediate states of a transaction are not visible to other transactions.

e.g. Two users updating the same account will not interfere with each other.

4. Durability : Once a transaction is committed, its changes are permanent, even in case of a system failure.

e.g. Money transferred is permanently recorded even if the system crashes immediately after.

c) Write step by step syntax to create, open and close cursor in PL/SQL.

Ans. 1. Declare cursor

```
CURSOR cursor_name IS
SELECT column1, column2
FROM table_name
WHERE condition;
```

2. Open cursor

OPEN cursor_name;

3. Fetch data from cursor

FETCH cursor_name INTO variable1, variable2;

4. Close cursor

CLOSE cursor_name;

Example :

DECLARE

v_emp_id employee.emp_id%TYPE;

v_emp_name employee.emp_name%TYPE;

CURSOR emp_cursor IS

SELECT emp_id,emp_name

FROM employee

WHERE dept_id = 10;

BEGIN

OPEN emp_cursor INTO v_emp_id,v_emp_name;

EXIT WHEN emp_cursor %NOTFOUND;

dbms_output.put_line(v_emp_id || ' - ' || v_emp_name);

END LOOP;

CLOSE emp_cursor;

END;

/

d) Explain savepoint and rollback command with example.

Ans. 1. SAVEPOINT : A savepoint is a named point in a transaction that can be "rolled back to" later, effectively undoing only the commands executed *after* that specific point, without affecting the work done before it.

Syntax :

SAVEPOINT savepoint_name;

Example :

SAVEPOINT A;

2. ROLLBACK : The ROLLBACK command is used to undo all work done in the current transaction, either back to the beginning or back to a specified SAVEPOINT.

Syntax :

ROLLBACK;

ROLLBACK TO savepoint_name;

Example :

ROLLBACK TO A;

a) Explain any four types of attributes with example.

Ans. 1. Simple Attributes : An attribute that cannot be broken down into smaller, meaningful components.

e.g.Attribute: Age

Reasoning: You cannot logically divide the value 25 into smaller parts that still represent the concept of age.

2. Composite Attributes : An attribute that can be divided into smaller, sub-attributes, each having its own independent meaning.

e.g. Attribute : FullName

Sub-Attributes : FirstName, MiddleInitial, LastName

Reasoning : While FullName is useful, it is composed of distinct pieces of data that might need to be queried or sorted individually.

3. Single-Valued Attributes : An attribute that holds only one value for a specific entity instance at any given time.

e.g. Attribute : EmployeeID

Reasoning : An employee can only have one unique identification number.

4. Multi-Valued Attributes : An attribute that can hold multiple values for a single entity instance. In a relational database, these often necessitate the creation of a separate table to maintain first normal form (1NF).

e.g. Attribute : PhoneNumbers

Values for one Person : A person might have a home phone, a work phone, and a cell phone, all stored under this single attribute concept.

b) Explain any four aggregate function with example.

Ans. 1. COUNT() : Returns the number of rows in a table or rows matching a condition.

e.g. SELECT COUNT(*) FROM employees;

2. SUM() : Returns the total sum of a numeric column.

e.g. SELECT SUM(salary) FROM employees;

3. AVG() : Returns the average value of a numeric column.

e.g. SELECT AVG(salary) FROM employees;

4. MAX() : Returns the maximum value of a numeric column.

e.g. SELECT MAX(salary) FROM employees;

c) Explain any four DML commands with syntax and example.

Ans. 1. SELECT : The SELECT command is the most frequently used DML operation. It is used to retrieve data from one or more tables in the database.

Syntax :

SELECT column1, column2 FROM table_name WHERE condition;

Example :

SELECT name, marks FROM Student WHERE marks > 50;

2. INSERT : The INSERT command is used to add a new row (record) into a table.

Syntax :

INSERT INTO table_name(column1, column2) VALUES(value1, value2);

Example :

INSERT INTO Student(name, marks) VALUES('Jay', 50);

3. UPDATE : The UPDATE command is used to modify existing data within the rows of a table.

Syntax :

UPDATE TABLE table_name SET column_name = 'new_value' WHERE condition;

Example :

UPDATE TABLE Student SET marks = '40' WHERE name = 'John';

4. DELETE : The DELETE command is used to **remove** existing rows (records) from a table.

Syntax :

DELETE FROM table_name WHERE condition;

Example :

DELETE FROM Student WHERE name = 'Jay';

d) Write a PL-SQL program to print odd numbers from 1 to 10.

Ans. DECLARE

i NUMBER;

BEGIN

FOR i in 1..10 LOOP

IF MOD(i,2) = 1 THEN

dbms_output.put_line(i);

END IF;

END LOOP;

END;

/

e) Explain the most common types of database failures.

Ans. 1. Transaction Failures (Logical Errors)

A transaction failure occurs when a running transaction cannot complete its execution successfully. This is often an internal issue related to the data or logic within the transaction itself.

Cause:

Logical Errors: An incorrect input or a coding error in a transaction causes it to abort (e.g., trying to divide by zero, or violating a CHECK constraint).

Deadlocks: Two or more transactions wait indefinitely for each other to release locks on resources. The DBMS detects the deadlock and aborts one or more transactions to resolve it.

2. System Failures (Software/Hardware Crashes)

A system failure involves a problem with the DBMS software or the underlying hardware that causes the computer system to stop suddenly.

Cause:

Software Failure: Bugs in the DBMS code, operating system crashes, or errors in application programs.

Hardware Failure: CPU, memory (RAM), or network adapter failures.

Power Failure: Sudden loss of power to the main memory, causing all volatile storage (RAM) contents to be lost.

3. Media Failures (Disk Failure)

A media failure, often considered the most serious type of failure, involves the destruction or corruption of the non-volatile storage where the database files physically reside.

Cause:

Head Crashes: Physical damage to the hard disk drive (HDD) reading heads.

Disk Controller Failure: Problems with the hardware or firmware managing the storage device.

Unreadable Data: Permanent corruption of blocks on the disk due to physical decay.

4. Catastrophic Failures (Disasters)

Catastrophic failures are widespread, major events that damage not only the database system but potentially the entire data center or computing environment.

Cause:

Natural Disasters (earthquakes, floods, fires).

Acts of Sabotage, terrorism, or war.

Infrastructure collapse (major utility failures).

6 marks

a) Explain strong and weak entity set with example. Draw ER diagram indicating strong and weak entity set.

Ans. 1. Strong Entity Set :

A Strong Entity Set is an entity that can be uniquely identified by its own attributes. It exists independently of any other entity in the schema.

Key Characteristic: It possesses a Primary Key.

Representation in ER Diagram: A single rectangle for the entity and a single line for the relationship. The primary key attribute is underlined.

2. Weak Entity Set :

A Weak Entity Set is an entity that cannot be uniquely identified by its own attributes alone. Its existence is dependent on a corresponding strong entity set.

Key Characteristic: It does not have a Primary Key of its own. It relies on the Primary Key of its parent (owner) strong entity set for unique identification.

Identifying Key: The combination of its own Partial Key (or Discriminator) and the Primary Key of the owner entity forms the Identifying Key.

Representation in ER Diagram: A double rectangle for the entity and a double diamond for the identifying relationship. The partial key is represented with a dashed underline.

b) Explain for loop syntax in PL-SQL with example of printing 10 to 1 reverse numbers.

Ans. Syntax :

```
FOR i in starting_number..exit_number LOOP
```

```
--statements to be executed
```

```
END LOOP;
```

Example :

```
DECLARE
i NUMBER;
BEGIN
FOR i in REVERSE 1..10 LOOP
dbms_output.put_line(i);
END LOOP;
END;
/
```

c) Explain function in PL-SQL with suitable example.

Ans. A PL/SQL Function is a named block of code that accepts zero or more input parameters, executes a set of SQL and PL/SQL statements, and must return a single value to the calling environment.

Syntax :

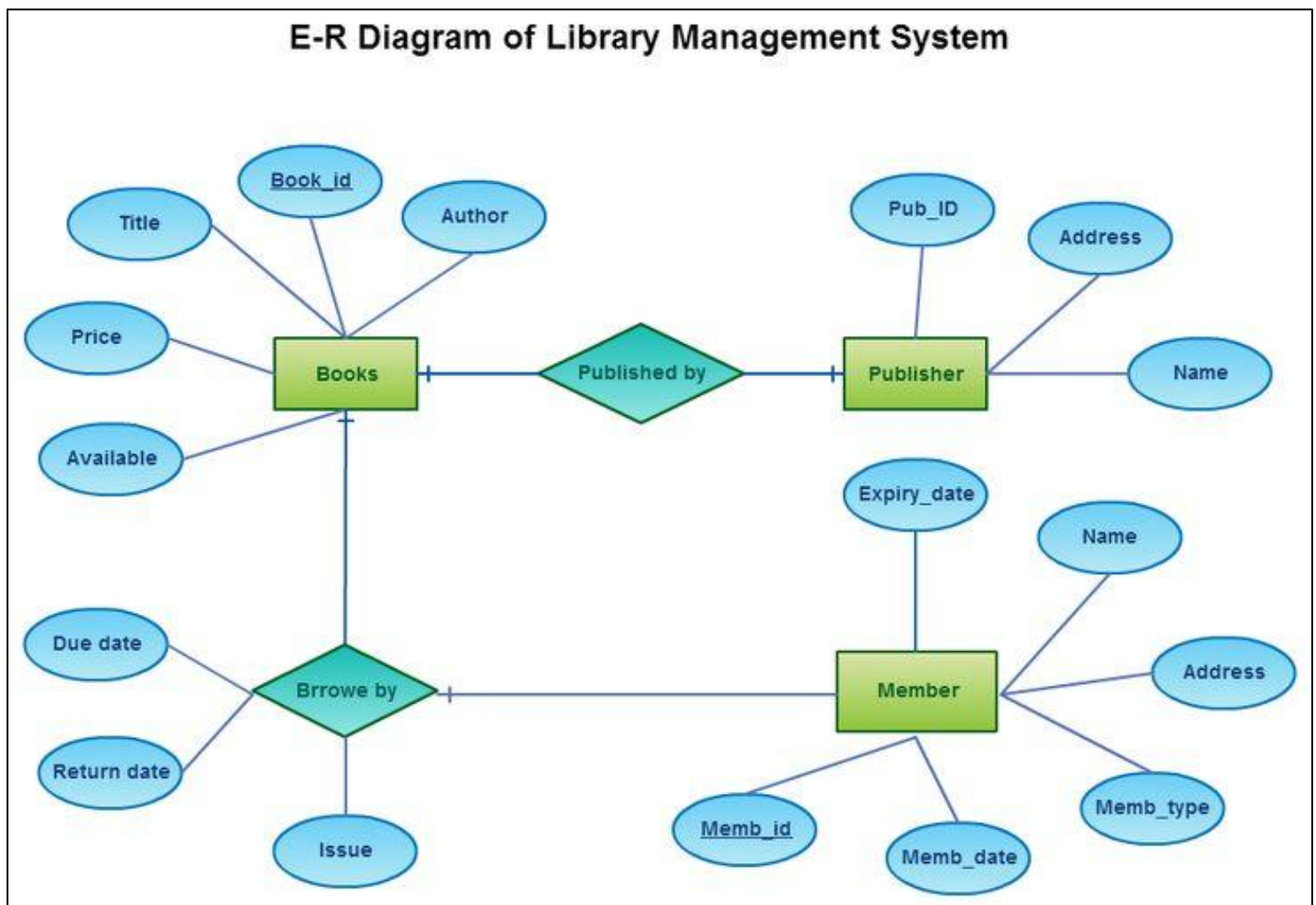
```
CREATE [OR REPLACE] FUNCTION function_name
(parameter1 [IN] datatype,
parameter2 [IN] datatype, ...)
RETURN return_datatype AS
-- Declaration section (local variables, exceptions)
BEGIN
-- Executable statements
RETURN expression; -- MUST return a value matching return_datatype
EXCEPTION
-- Exception handlers
END [function_name];
/
```

Example :

```
CREATE FUNCTION add_num(a IN NUMBER, b IN NUMBER)
RETURN NUMBER
AS
v_result NUMBER;
BEGIN
v_result := a+b;
RETURN v_result;
END add_num;
/
DECLARE
v_sum NUMBER;
BEGIN
v_sum := add_num(10,20);
dbms_output.put_line('Sum :- '||sum);
END;
/
```

a) Draw an ER diagram of library management system considering publisher, books, member. Also show primary key, weak entity and strong entity.

Ans.



b) State and explain 1 NF and 2 NF with example.

Ans. 1NF (First Normal Form) :

Definition :

A relation (table) is said to be in First Normal Form (1NF) if each column contains only atomic (indivisible) values and each record is unique.

Rules of 1NF:

Each cell should contain only a single value (no multiple or repeating values).

Each record (row) must be unique.

The order of columns and rows does not matter.

Example (Before 1NF):

<i>Student_ID</i>	<i>Name</i>	<i>Subjects</i>
1	Raj	DBMS, OS
2	Priya	C, Java, Python

Here, the column Subjects has multiple values, violating 1NF.

After converting to 1NF:

<i>Student_ID</i>	<i>Name</i>	<i>Subject</i>
1	Raj	DBMS
1	Raj	OS
2	Priya	C
2	Priya	Java
2	Priya	Python

Now, every field has a single (atomic) value, so the table is in 1NF.

2NF (Second Normal Form):

Definition:

A table is in Second Normal Form (2NF) if:

It is already in 1NF, and

Every non-key attribute is fully functionally dependent on the entire primary key.

(There should be no partial dependency.)

Note:

Partial dependency means that a non-key attribute depends on part of a composite (multi-attribute) primary key.

Example (Before 2NF):

<i>Student_ID</i>	<i>Subject</i>	<i>Student_Name</i>	<i>Marks</i>
1	DBMS	Raj	85
1	OS	Raj	90
2	Java	Priya	88

Primary Key: (Student_ID, Subject)

Here,

Marks depends on both Student_ID and Subject → OK

Student_Name depends only on Student_ID → Partial dependency exists

After converting to 2NF:

Table 1: Student

<i>Student_ID</i>	<i>Student_Name</i>
1	Raj
2	Priya

Table 2: Marks

<i>Student_ID</i>	<i>Subject</i>	<i>Marks</i>
1	DBMS	85
1	OS	90
2	Java	88

Now, there is no partial dependency, so the database is in 2NF.

c) Explain any two types of join with example.

Ans. 1. INNER JOIN

The INNER JOIN is the most common type of join. It returns only the rows that have **matching values** in the specified join column(s) in **both tables**. Any rows that do not have a match in the other table are excluded from the result set.

Syntax :

SELECT columns

FROM TableA

INNER JOIN TableB

ON TableA.common_column = TableB.common_column;

Example :

SELECT E.Name, D.DepartmentName

FROM Employees E

INNER JOIN Departments D

ON E.DepartmentID = D.DepartmentID;

2. LEFT JOIN (or LEFT OUTER JOIN)

The LEFT JOIN returns **all rows** from the **left table** (the first table specified in the FROM clause) and the **matching rows** from the **right table**. If there is no match for a row in the left table, NULL is returned for all columns from the right table.

Syntax :

SELECT columns

FROM TableA

LEFT JOIN TableB

ON TableA.common_column = TableB.common_column;

Example :

SELECT E.Name, D.DepartmentName

FROM Employees E

LEFT JOIN Departments D

ON E.DepartmentID = D.DepartmentID;